1 July 28, 2025 (Class 1)

Course Logistics

2 July 29, 2025 (Class 2)

Paper reading: Nature Review "Deep Learning", LeCun et al.

3 August 1, 2025 (Class 3)

Continued paper reading: Nature Review "Deep Learning", LeCun et al.

4 August 4, 2025 (Class 4)

4.1 Supervised Learning Setting

4.1.1 Data

Let $\underline{x} \in \mathcal{X}$ be the input data, where $\mathcal{X}$ is the input space, and $y \in \mathcal{Y}$ is the corresponding label, where $\mathcal{Y}$ is the label space. We define

$$\mathcal{D}_{train} = (\underline{x}_i^t, y_i^t)_{i=1}^N$$
$$\mathcal{D}_{val} = (\underline{x}_i^v, y_i^v)_{i=1}^M$$

Setting : $(\underline{x}, y) \in p(\underline{x}, y)$, where p is a fixed but unknown distribution over $\mathcal{X} \times \mathcal{Y}$.

4.1.2 Model/Machine

A parameterized function $f(\underline{x}; \theta)$ that maps an input data point $\underline{x}$ to a label $\widehat{y}$.

$$f : \mathcal{X} \times \mathcal{H} \to \mathcal{Y}$$

where $\mathcal{H}$ is the parameter space, and θ are the parameters of the model.

$$f(\underline{x} : \theta) = f_L^{(\theta_L)} \circ f_{L-1}^{(\theta_{L-1})} \circ \ldots \circ f_1^{(\theta_1)}$$

In several settings, we have a template or a building block model.

Multi-Layer Perceptron (MLP)

$$f = \sigma(\underline{W}\underline{x} + \underline{b})$$

where σ is a non-linear activation function, $\underline{W}$ is the weight matrix, and $\underline{b}$ is the bias vector.

Convolutional Neural Network (CNN)

$$f = \sigma(\text{Conv}(\underline{W}, \underline{x}) + \underline{B})$$

where σ is a non-linear activation function, Conv is the convolution operation, $\underline{W}$ is the filter/kernel, and $\underline{B}$ is the bias term.

Transformer

$$f = \text{Transformer}(\underline{\text{x}}, \theta)$$

Recurrent Neural Network (RNN)

$$f^t = \sigma(\underline{\text{W}} \cdot \underline{\text{x}}^t + R \cdot f^{t-1})$$

where σ is a non-linear activation function, $\underline{\text{W}}$ is the weight matrix, R is the recurrent weight matrix, and f^{t-1} is the output from the previous time step.

4.1.3 Optimization

We want to find the parameters θ such that the model

- Low training loss, i.e. on $\mathcal{D}_{train}$
- Generalizes well, i.e. low validation loss on $\mathcal{D}_{val}$

Let per sample distance be defined as

$$d : \underline{\text{y}} \times \underline{\text{y}} \to \mathbb{R}$$

Let the loss $\mathcal{L}(\theta)$ be defined as

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} d(f(\underline{\text{x}}_i; \theta), y_i)$$

We want to find the optimal parameters θ^* such that

$$\theta^* = \arg\min_{\theta \in \mathcal{H}} \mathcal{L}(\theta)$$

5 August 5, 2025 (Class 5)

6 August 8, 2025 (Class 6)

7 August 11, 2025 (Class 7)

8 August 12, 2025 (Class 8)

9 August 18, 2025 (Class 9)

10 August 19, 2025 (Class 10)

11 August 21, 2025 (Class 11)

12 August 25, 2025 (Class 12)

13 August 26, 2025 (Class 13)

14 August 29, 2025 (Class 14)

15 September 1, 2025 (Class 15)

16 September 2, 2025 (Class 16)

17 September 4, 2025 (Class 17)

17.1 Momentum

The momentum update rule is defined as,

$$\begin{aligned} &\upsilon \leftarrow \alpha\upsilon - \epsilon\nabla_\theta \left(\frac{1}{m} \sum_{i=1}^{m} L\left(f(x^{(i)}; \theta), y^{(i)} \right) \right) \\ &\theta \leftarrow \theta + \upsilon. \end{aligned}$$

17.2 Nesterov Momentum

The Nesterov momentum rule is defined as,

$$\begin{aligned} &\upsilon \leftarrow \alpha\upsilon - \epsilon\nabla_\theta \left(\frac{1}{m} \sum_{i=1}^{m} L\left(f(x^{(i)}; \theta + \alpha\upsilon), y^{(i)} \right) \right) \\ &\theta \leftarrow \theta + \upsilon. \end{aligned}$$

Algorithm 1 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ, momentum parameter α.
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.
while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

18 September 8, 2025 (Class 18)

19 September 9, 2025 (Class 19)

20 AdaGrad (Adaptive Gradient)

Algorithm 2 The AdaGrad algorithm

Require: Global learning rate ϵ
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, perhaps 10^{-7}, for numerical stability
Initialize gradient accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$
    Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. (Division and square root applied element-wise)
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

20.1 RMSProp

20.2 Adam (Adaptive Moments)

20.3 Generative Adversarial Networks

See [Generative Adversarial Networks].

20.4 Variational Autoencoders

See [Variational Autoencoders]

Algorithm 3 The RMSProp algorithm

Require: Global learning rate ϵ, decay rate ρ.
Require: Initial parameter $\boldsymbol{\theta}$
Require: Small constant δ, usually 10^{-6}, used to stabilize division by small numbers.
Initialize accumulation variables $\boldsymbol{r} = 0$
while stopping criterion not met **do**
  Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
  Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
  Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \boldsymbol{g} \odot \boldsymbol{g}$
  Compute parameter update: $\Delta \boldsymbol{\theta} = -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
  Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$
end while

Algorithm 4 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)
Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)
Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})
Require: Initial parameters $\boldsymbol{\theta}$
Initialize 1st and 2nd moment variables $\boldsymbol{s} = \boldsymbol{0}, \boldsymbol{r} = \boldsymbol{0}$
Initialize time step $t = 0$
while stopping criterion not met **do**
  Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
  Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
  $t \leftarrow t + 1$
  Update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1) \boldsymbol{g}$
  Update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2) \boldsymbol{g} \odot \boldsymbol{g}$
  Correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
  Correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
  Compute update: $\Delta \boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
  Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$
end while

20.4.1 See also

20.4.2 References